ECEN 4/5632 Theory and Application of Digital Filters
Lecture 13, 10-20-26, 10-22-26
Peter Mathys*

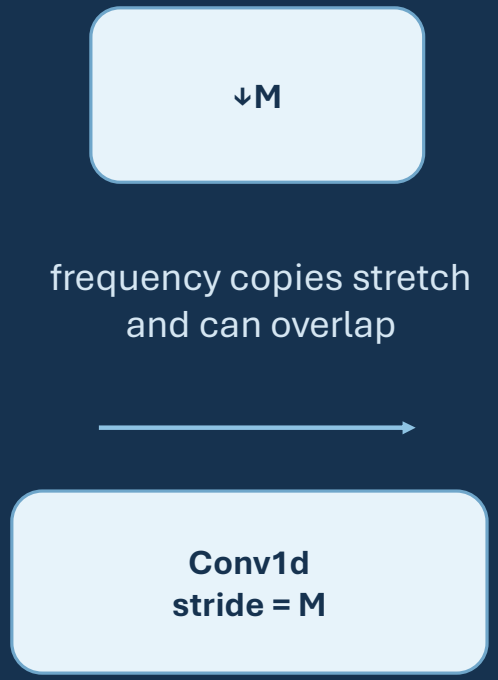


* With generous help from Gemini Pro and ChatGPT

WEEK 8 ● LECTURE 13

Downsampling, Decimation, and CNN Strides

Classical sampling theory meets modern convolutional networks



Oppenheim & Schafer §4.6 | Raschka: CNN architecture

Learning objectives

By the end of 75 minutes, students should be able to reason in both time and frequency domains.

- Compute **downsampled sequences** from x[n] and M.
- Predict **spectral stretching and aliasing** using the DTFT relation.
- Design the **anti-aliasing stage** of a decimator.
- Explain why **stride > 1** in a CNN is a sampling operation.
- Diagnose **shift-equivariance failures** caused by aliased feature maps.

LPF for antialias

Pacing

0–25 min: downsampling
25–50 min: decimation
50–75 min: CNN strides

Core question

What must happen before we throw samples away?

Shift equivariance:
Shift input to a system by small amount ⇒ Expect output of system to be shifted by a comparable and predictable small amount

01

Downsampling in time

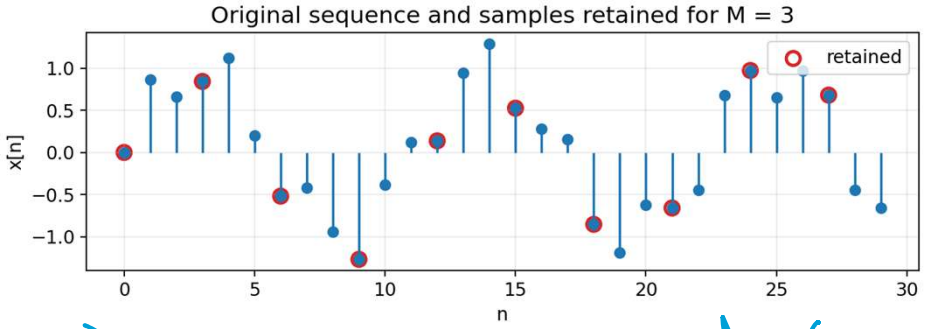
Keep every Mth sample; the sample spacing grows by M.

0:04–0:10

Downsampling: the time-domain definition

$$x_d[n] = x[nM]$$

- Input sample rate: F_s.
- Output sample rate: F_s/M.
- No new values are computed—the operation selects a subsequence.
- Information loss is harmless only when the discarded samples are redundant relative to the signal bandwidth.

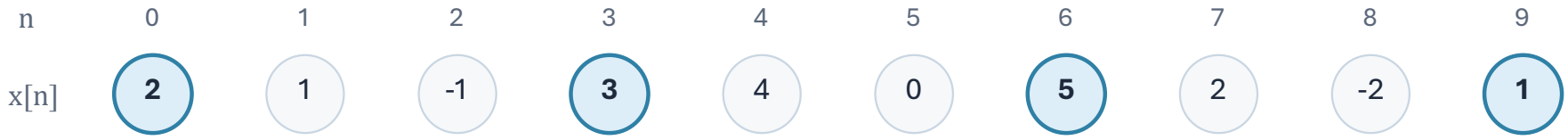


$x = \sin(2\pi f_a \cdot n) + 0.35 \sin(2\pi f_b n)$ $F_s = 9000$ samples/sec

0:10–0:14

A quick index-mapping example

Suppose x[n] = { 2, 1, −1, 3, 4, 0, 5, 2, −2, 1, … } and M = 3.



Result

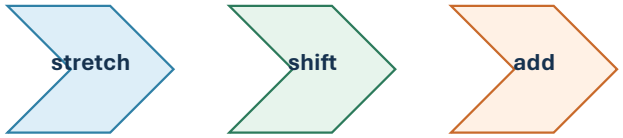
x_d[n] = { 2, 3, 5, 1, … }

0:14–0:20

What happens in frequency?

$$X_d(e^{j\omega}) = \frac{1}{M} \sum_{k=0}^{M-1} X\left(e^{j(\omega - 2\pi k)/M}\right)$$

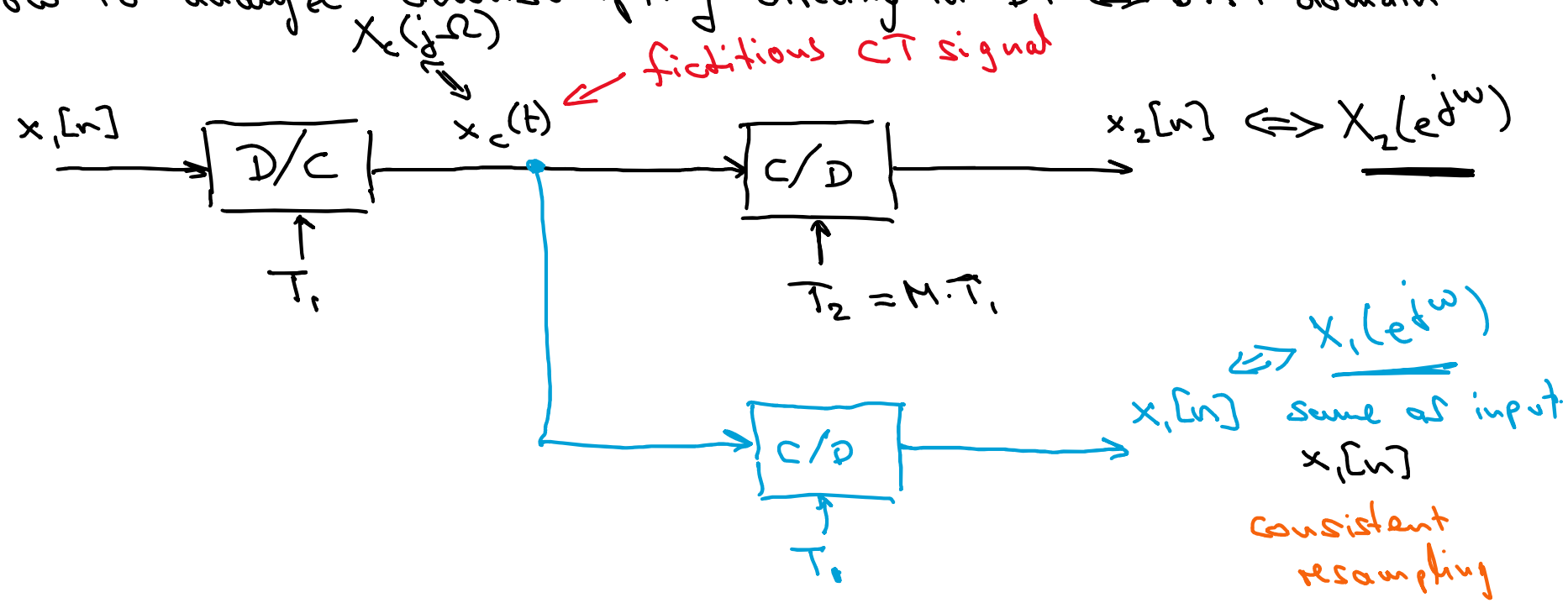
- **Stretch:** each input spectrum is evaluated at ω/M, so features expand by M on the output frequency axis.
- **Shift:** M periodically shifted copies appear.
- **Add:** the copies are summed; overlap is irreversible aliasing.
- **Scale:** the 1/M factor accounts for the rate change in the DTFT relation.



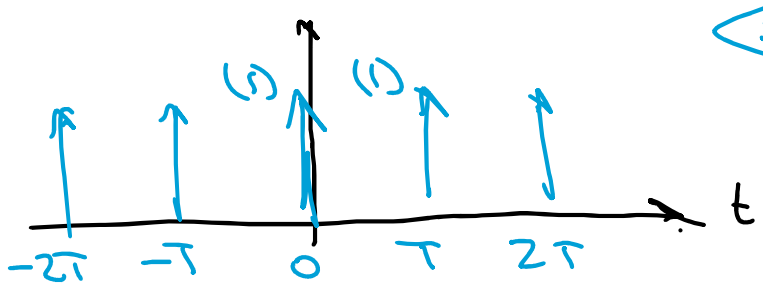
Interpretation

Downsampling is a sampling operation performed on an already-discrete signal.

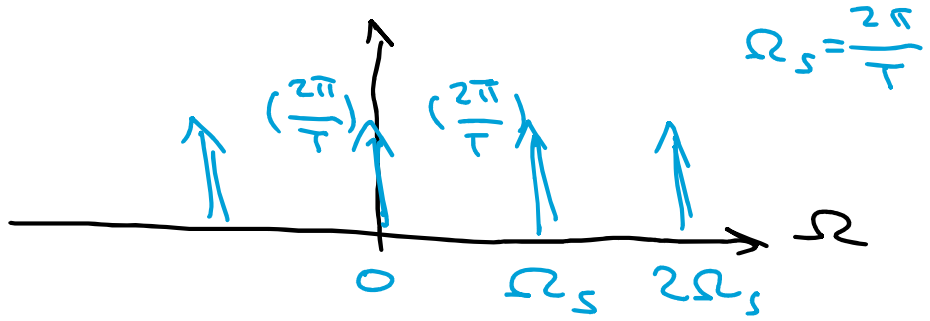
How to analyze downsampling directly in DT $\Leftrightarrow$ DTFT domain



$$s(t) = \sum_n \delta(t - nT) \qquad S(j\Omega) = \frac{2\pi}{T} \sum_k \delta(\Omega - k\Omega_s)$$



$\Longleftrightarrow$



$$x_s(t) = x_c(t) \cdot \sum_n \delta(t - nT)$$
$$= \sum_n x_c(nT) \cdot \delta(t - nT)$$
$$= \sum_n x[n]\, \delta(t - nT)$$

$x[n] = x_c(nT)$

$$X_s(j\Omega) = \frac{1}{2\pi} X_c(j\Omega) * S(j\Omega)$$
$$= \frac{1}{T} \sum_k X_c(j(\Omega - k\Omega_s))$$

$\Longleftrightarrow$

$$X_s(j\Omega) = \mathcal{F}\left\{\sum_n x[n]\, \delta(t - nT)\right\}$$
$$= \sum_n x[n]\, \mathcal{F}\{\delta(t - nT)\}$$
$$= \sum_n x[n]\, e^{-j\Omega T n} = X(e^{j\omega})\Big|_{\omega = \Omega T} = X(e^{j\Omega T})$$

$$X_s(j\Omega) = \mathcal{F}\left\{\sum_n x[n]\,\delta(t-nT)\right\}$$
$$= \sum_n x[n]\,\mathcal{F}\{\delta(t-nT)\}$$
$$= \sum_n x[n]\,e^{-j\Omega T n} = X(e^{j\omega})\Big|_{\omega=\Omega T} = X(e^{j\Omega T})$$

$$X_s(j\Omega) = \frac{1}{T}\sum_k X_c(j(\Omega - k\Omega_s))$$

$$= X(e^{j\Omega T}) = X(e^{j\omega})\Big|_{\omega=\Omega T} \Rightarrow \Omega = \frac{\omega}{T}$$

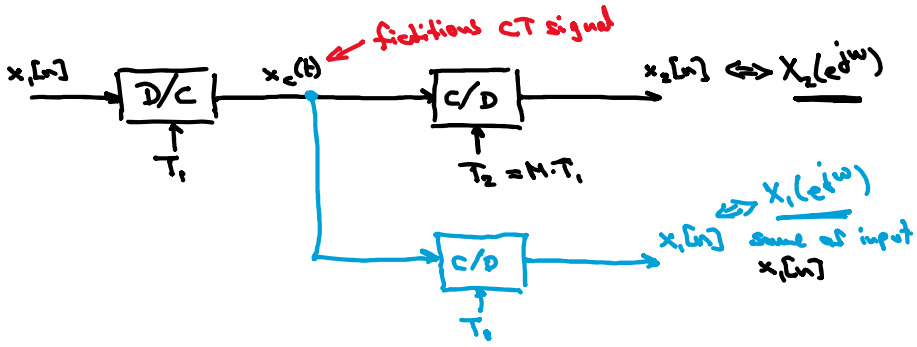
$$X_s(j\Omega) = \frac{1}{2\pi} X_c(j\Omega) * S(j\Omega)$$
$$= \frac{1}{T}\sum_k X_c(j(\Omega - k\Omega_s))$$

$$\boxed{X(e^{j\Omega T}) = \frac{1}{T}\sum_k X_c(j(\Omega - k\Omega_s))}$$

$$\boxed{X(e^{j\omega}) = \frac{1}{T}\sum_k X_c\left(j\left(\frac{\omega}{T} - \frac{2\pi k}{T}\right)\right)}$$

$$X(e^{j\Omega T}) = \frac{1}{T}\sum_k X_c\left(j(\Omega - k\Omega_s)\right)$$

$$X(e^{j\omega}) = \frac{1}{T}\sum_k X_c\left(j\left(\frac{\omega}{T} - \frac{2\pi k}{T}\right)\right)$$



← fictitious

$$X_1(e^{j\omega_1}) = \frac{1}{T_1}\sum_k X_c\left(j\left(\frac{\omega_1}{T_1} - \frac{2\pi k}{T_1}\right)\right) \qquad \omega_1 = \Omega T_1$$

$$X_2(e^{j\omega_2}) = \frac{1}{T_2}\sum_k X_c\left(j\left(\frac{\omega_2}{T_2} - \frac{2\pi k}{T_2}\right)\right) \qquad \omega_2 = \Omega T_2$$

$T_2 = MT_1$

:(we down sample by M

$$= \frac{1}{MT_1}\sum_l X_c\left(j\left(\frac{\omega_2}{MT_1} - \frac{2\pi l}{MT_1}\right)\right)$$

Let $l = i + kM$ $\quad i = 0, 1, \ldots, M-1$ $\quad -\infty < k < \infty$

$$X_2(e^{j\omega_2}) = \frac{1}{M}\sum_{i=0}^{M-1}\frac{1}{T_1}\sum_k X_c\left(j\left(\frac{\omega_2}{MT_1} - \frac{2\pi i}{MT_1} - \frac{2\pi kM}{MT_1}\right)\right)$$

$$= \frac{1}{M}\sum_{i=0}^{M-1}\underbrace{\frac{1}{T_1}\sum_k X_c\left(j\left(\frac{\omega_2 - 2\pi i}{M} - 2\pi k\right)/T_1\right)}_{= X_1(e^{j(\omega_1 - 2\pi i)/M})}$$

$$X_1(e^{j\omega_1}) = \frac{1}{T_1} \sum_k X_c\left(j\left(\frac{\omega_1}{T_1} - \frac{2\pi k}{T_1}\right)\right)$$

← fictitious

$\omega_1 = \Omega T_1$

$$X_2(e^{j\omega_2}) = \frac{1}{M} \sum_{i=0}^{M-1} \frac{1}{T_1} \sum_k X_c\left(j\left(\frac{\omega_2}{MT_1} - \frac{2\pi i}{MT_1} - \frac{2\pi k M}{MT_1}\right)\right)$$

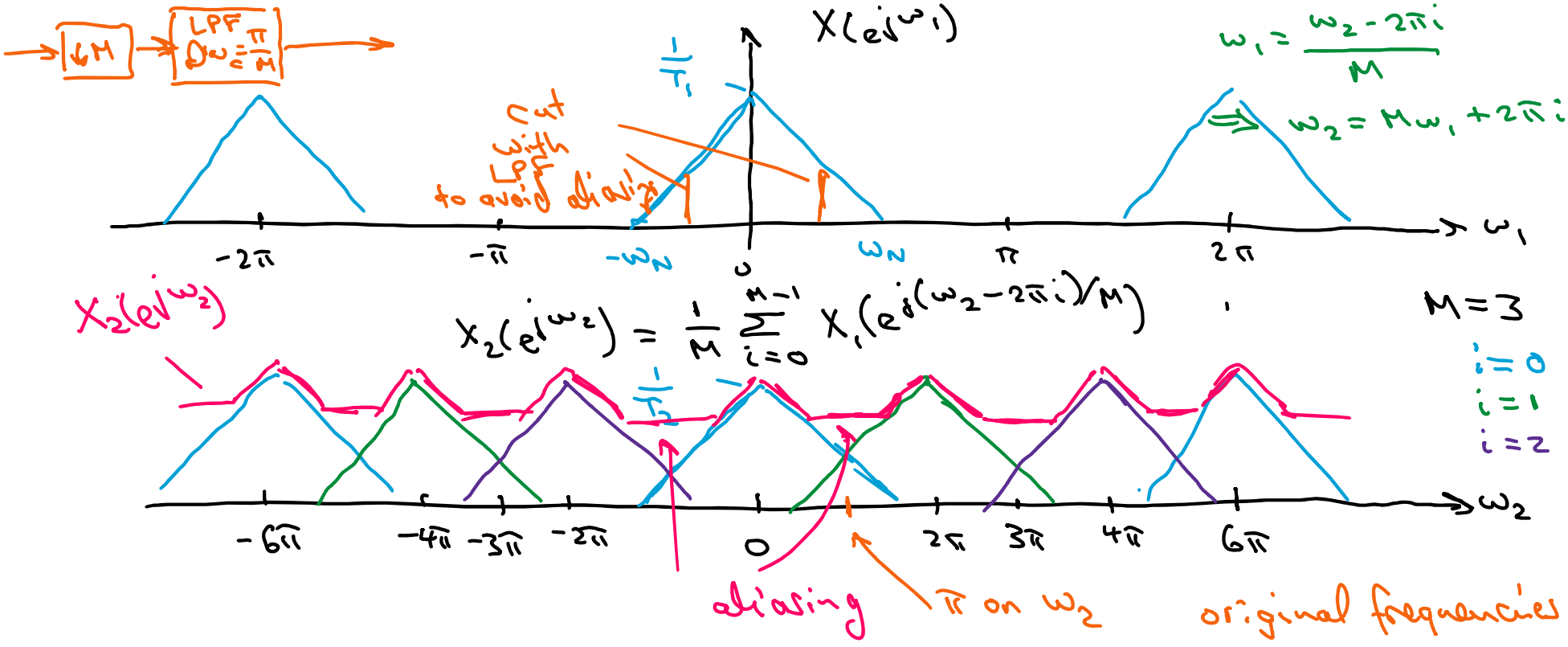
$$= \frac{1}{M} \sum_{i=0}^{M-1} \frac{1}{T_1} \sum_k X_c\left(j\left(\frac{\omega_2 - 2\pi i}{M} - 2\pi k\right)/T_1\right)$$

$$= X_1\left(e^{j(\omega_1 - 2\pi i)/M}\right)$$

$$= \frac{1}{M} \sum_{i=0}^{M-1} X_1\left(e^{j(\omega_1 - 2\pi i)/M}\right)$$

DTFT of down sampled signal by integer M

DTFT of original signal



original frequencies

$\omega_2 < \pi$ on ω_2

or $\omega_2 < \frac{\pi}{M}$ on ω_1

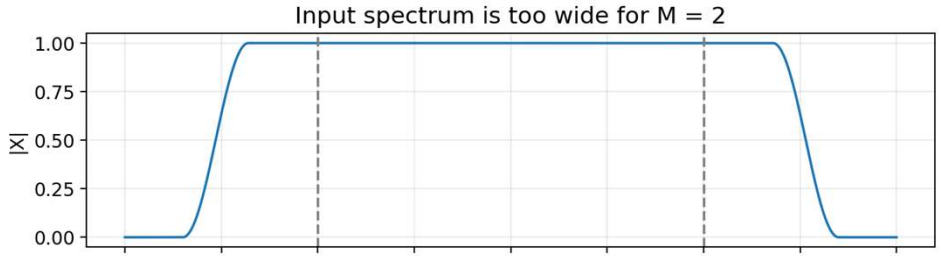
in general: to avoid aliasing

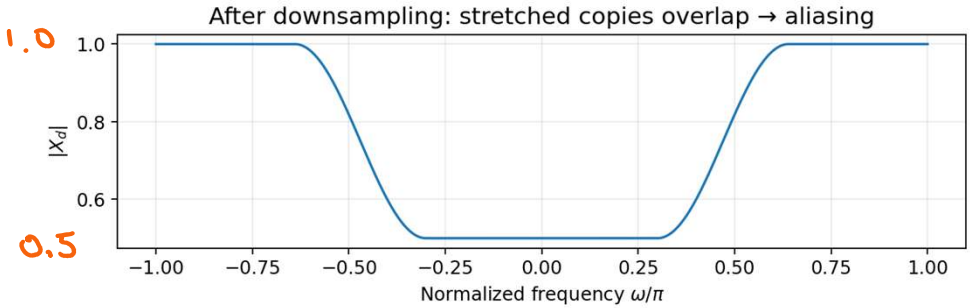
we need $\omega_2 < \frac{\pi}{M}$

0:20–0:25

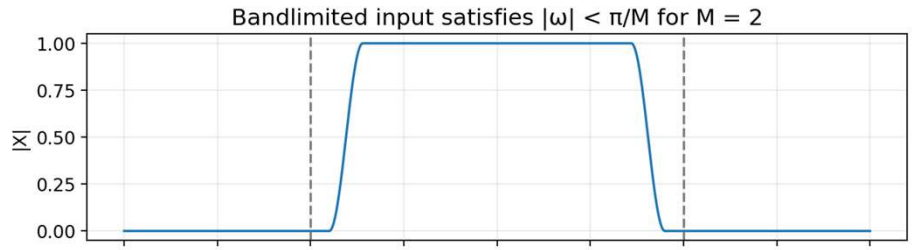
Aliasing: when stretched copies overlap

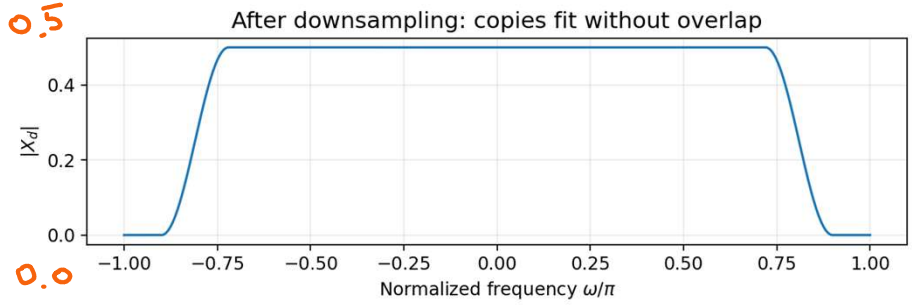
does have aliasing





no aliasing





Alias-free requirement

Before ↓M, confine the signal to $|\omega| < \pi/M$ (with a practical transition band).

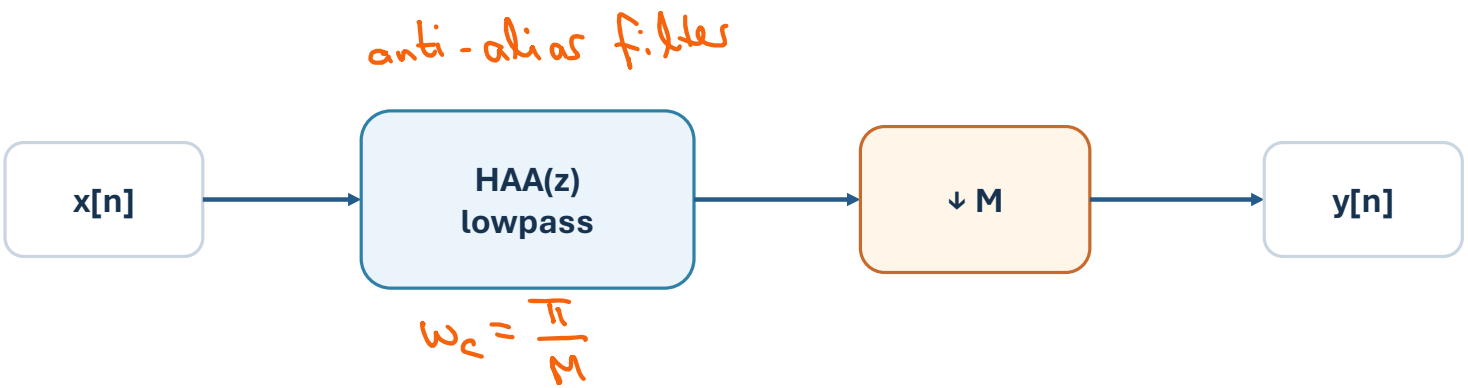
02

Decimation = filter + downsample

The lowpass filter removes content that would fold into the preserved band.

0:25–0:32

Why the anti-aliasing filter is necessary



Ideal cutoff

ω_c = π/M

Practical filter

Choose passband edge < π/M and stopband edge ≤ π/M.

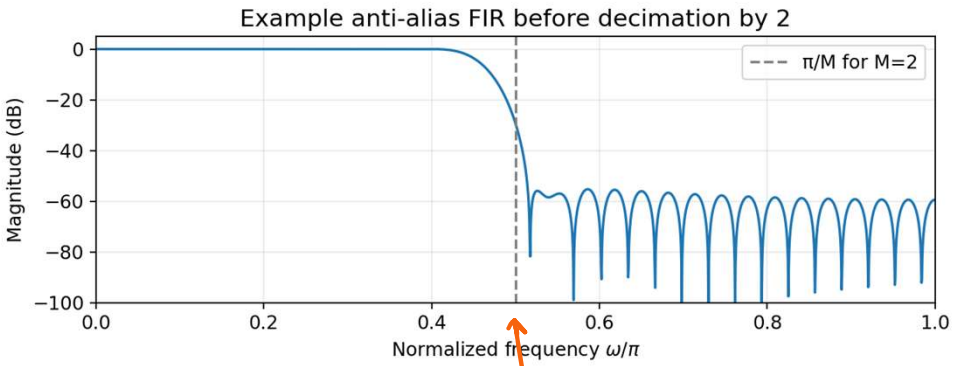
Design tradeoff

Sharper transition ⇒ longer FIR ⇒ more delay and computation.

0:32–0:39

Filter design example: decimate by 2

M = 2



$\frac{\omega_c}{\pi} = \frac{1}{M} = \frac{1}{2}$

- Output Nyquist limit corresponds to input |ω| = π/2.
- Preserve the useful band below the transition region.
- Suppress energy above π/2 before ↓2.
- A linear-phase FIR gives constant group delay in the passband.

Implementation note

scipy.signal.decimate or resample_poly handles filtering + rate change for you—but the design assumptions still matter.

```
scipy.signal.decimate(x, q=M, n=None, ftype='iir', zero_phase=True)
```

n = filter order

ftype = 'iir' or 'fir'

0:39–0:46

Why linear-phase FIR is a common choice

Preserved waveform shape

Constant group delay keeps relative phase relationships across the passband.

Stable by construction

Finite impulse response filters have no recursive poles to destabilize the decimator.

Predictable delay

Symmetric taps give delay (N−1)/2 samples at the input rate.

relatively large compared to IIR

Efficient structures

Polyphase implementations avoid computing samples that will be discarded.

Important nuance: linear phase is useful, not mandatory. Minimum-phase or IIR designs may be preferred when latency dominates.

Concept check: safe decimation

A signal occupies $|\omega| \le 0.22\pi$. What is the largest integer M that can be used without aliasing, assuming an ideal anti-alias filter that preserves the whole band?

Reveal after discussion

$0.22\pi \le \frac{\pi}{M}$ $\quad M \le \frac{1}{0.22} \approx 4.5$ $\quad \lfloor 4.5 \rfloor = 4$ is max M

03

CNN strides are downsampling

A strided convolution combines learned filtering with sample selection.

0:50–0:57

Conv1d with stride M: the DSP view



$$y[n] = \sum_k h[k]x[nM - k]$$

Key difference from classical decimation

The kernel is learned for task loss—not explicitly constrained to be a lowpass anti-aliasing filter.

Correlation ("convolution") as matrix-vector product

Example: kernel $w[n] = [w[0], w[1], w[2]]$

$$y[n] = \sum_k w[k]\, x[n+k] = \sum_{k=0}^{2} w[k] \cdot x[n+k]$$

$$y[0] = w[0]x[0] + w[1]\cdot x[1] + w[2]\cdot x[2]$$
$$y[1] = w[0]x[1] + w[1]\cdot x[2] + w[2]x[3]$$
$$y[2] = w[0]x[2] + w[1]x[3] + w[2]x[4]$$

$$\underset{(4\times 1)}{y} = \underset{(4\times 6)}{W}\ \underset{(6\times 1)}{x}$$

stride 1

$$\begin{bmatrix} y[0] \\ y[1] \\ y[2] \\ y[3] \end{bmatrix} = \underbrace{\begin{bmatrix} w[0] & w[1] & w[2] & 0 & 0 & 0 \\ 0 & w[0] & w[1] & w[2] & 0 & 0 \\ 0 & 0 & w[0] & w[1] & w[2] & 0 \\ 0 & 0 & 0 & w[0] & w[1] & w[2] \end{bmatrix}}_{W} \cdot \begin{bmatrix} x[0] \\ x[1] \\ x[2] \\ x[3] \\ x[4] \\ x[5] \end{bmatrix}$$

stride 1

$$\begin{bmatrix} y[0] \\ y[1] \\ y[2] \\ y[3] \end{bmatrix} = \underbrace{\begin{bmatrix} w[0] & w[1] & w[2] & 0 & 0 & 0 \\ 0 & w[0] & w[1] & w[2] & 0 & 0 \\ 0 & 0 & w[0] & w[1] & w[2] & 0 \\ 0 & 0 & 0 & w[0] & w[1] & w[2] \end{bmatrix}}_{W} \cdot \begin{bmatrix} x[0] \\ x[1] \\ x[2] \\ x[3] \\ x[4] \\ x[5] \end{bmatrix}$$

$$\vec{y} = W \cdot \vec{x}$$

stride 2

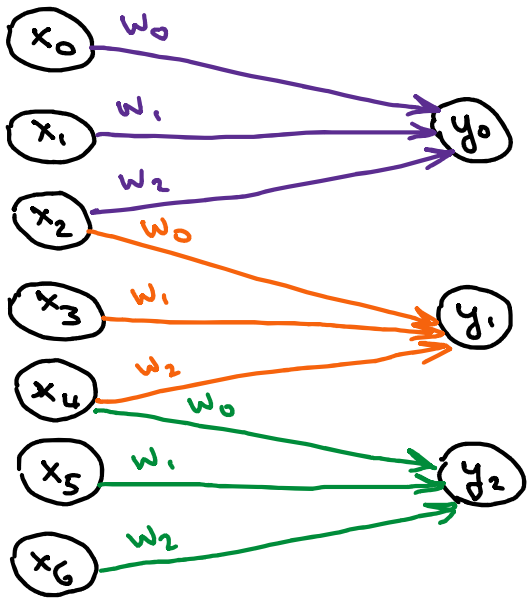
stride

$$\begin{bmatrix} y[0] \\ y[1] \\ y[2] \end{bmatrix} = \underbrace{\begin{bmatrix} w[0] & w[1] & w[2] & 0 & 0 & 0 & 0 \\ 0 & 0 & w[0] & w[1] & w[2] & 0 & 0 \\ 0 & 0 & 0 & 0 & w[0] & w[1] & w[2] \end{bmatrix}}_{\text{new } W} \begin{bmatrix} x[0] \\ x[1] \\ x[2] \\ x[3] \\ x[4] \\ x[5] \\ x[6] \end{bmatrix}$$

$$\underset{(3\times 1)}{y} = \underset{(3\times 7)}{W} \cdot \underset{(7\times 1)}{x}$$

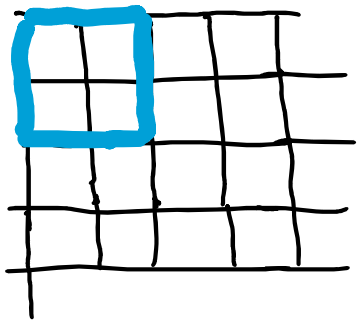
Visualizing Stride 2 as a Weighted Network

$$\begin{bmatrix} y[0] \\ y[1] \\ y[2] \end{bmatrix} = \underbrace{\begin{bmatrix} w[0] & w[1] & w[2] & 0 & 0 & 0 & 0 \\ 0 & 0 & w[0] & w[1] & w[2] & 0 & 0 \\ 0 & 0 & 0 & 0 & w[0] & w[1] & w[2] \end{bmatrix}}_{\text{new } W} \begin{bmatrix} x[0] \\ x[1] \\ x[2] \\ x[3] \\ x[4] \\ x[5] \\ x[6] \end{bmatrix}$$

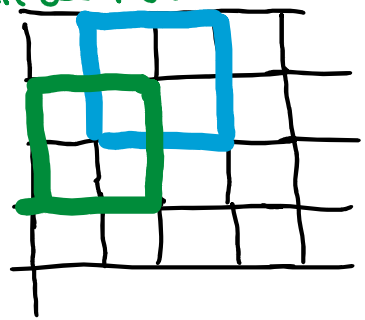


Strides in CNNs

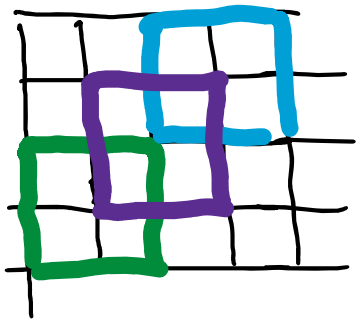
2d images processing with 2 × 2 kernel



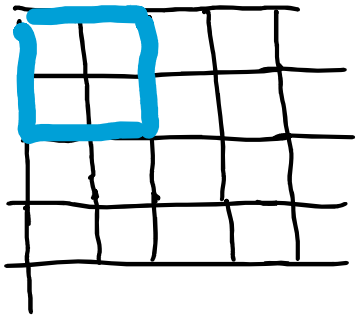
stride 1 to the right
stride 1 down

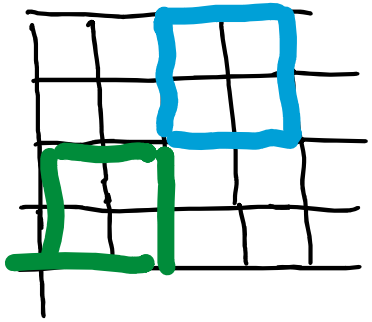


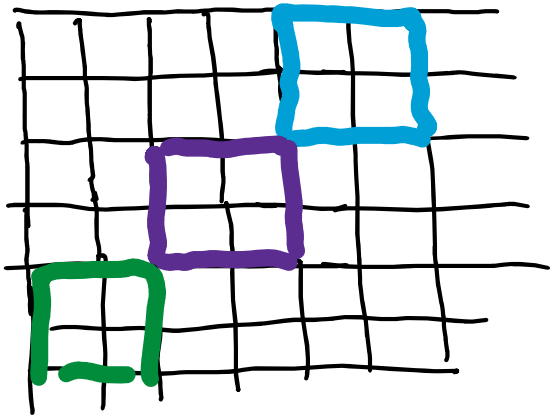
stride 1 diagonal



stride 2







Filtering and Downsampling with numpy, weight matrix W and Conv1d

```python
# Use deterministic kernel to make comparisons transparent.
x_np = np.array([0., 1., 2., 1., 0., -1., -2., -1., 0.], dtype=np.float32)
h_np = 1/4*np.array([1, 2, 1], dtype=np.float32)
#h_np = np.array([1, 0, 0], dtype=np.float32)
#h_np = np.array([0, 1, 0], dtype=np.float32)
stride = 2
```

```python
# NumPy: correlation convention to match PyTorch Conv1d.
full_valid = np.correlate(x_np, h_np, mode="valid")
y_numpy = full_valid[::stride]
```

```python
# Weight matrix multiply
m = int(1 + np.ceil((x_np.size - h_np.size) / stride))  # number of rows
W = np.zeros((m, (m-1)*stride + h_np.size))
for i in range(m):
    W[i, i*stride:i*stride + h_np.size] = h_np
W = W[:,:x_np.size]
print(f'W =\n{W}')
y_W = W @ x_np
```

```
W =
[[0.25 0.5  0.25 0.   0.   0.   0.   0.   0.  ]
 [0.   0.   0.25 0.5  0.25 0.   0.   0.   0.  ]
 [0.   0.   0.   0.   0.25 0.5  0.25 0.   0.  ]
 [0.   0.   0.   0.   0.   0.   0.25 0.5  0.25]]
```

Stride 2 (Downsample with $M = 2$)
Size 3 Kernel

```python
# PyTorch Conv1d
conv = nn.Conv1d(1, 1, kernel_size=h_np.size, stride=stride, bias=False)
with torch.no_grad():
    conv.weight[:] = torch.tensor(h_np).view(1, 1, -1)

x_t = torch.tensor(x_np).view(1, 1, -1)
y_torch = conv(x_t).detach().numpy().squeeze()
```

```python
# Compare results
print(f'Numpy:    y = {y_numpy}')
print(f'W matrix: y = {y_W}')
print(f'PyTorch:  y = {y_torch}')
```

```
Numpy:    y = [ 1.  1. -1. -1.]
W matrix: y = [ 1.  1. -1. -1.]
PyTorch:  y = [ 1.  1. -1. -1.]
```

```python
print(f'x = {x_np}')
```

```
x = [ 0.  1.  2.  1.  0. -1. -2. -1.  0.]
```

Downsampling Without Filtering, hurting Shift Equivariance

```
# Use deterministic kernel to make comparisons transparent.
x_np = np.array([0., 1., 2., 1., 0., -1., -2., -1., 0.], dtype=np.float32)
#h_np = 1/4*np.array([1, 2, 1], dtype=np.float32)
#h_np = np.array([1, 0, 0], dtype=np.float32)
h_np = np.array([0, 1, 0], dtype=np.float32)
stride = 2
```

Stride 2 (Downsample with $M = 2$)
Size 1 Kernel, Phase 1

```
# NumPy: correlation convention to match PyTorch Conv1d.
full_valid = np.correlate(x_np, h_np, mode="valid")
y_numpy = full_valid[::stride]
```

```
# Weight matrix multiply
m = int(1 + np.ceil((x_np.size - h_np.size) / stride))  # number of rows
W = np.zeros((m, (m-1)*stride + h_np.size))
for i in range(m):
    W[i, i*stride:i*stride + h_np.size] = h_np
W = W[:,:x_np.size]
print(f'W =\n{W}')
y_W = W @ x_np
```

```
W =
[[0. 1. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 1. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 1. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 1. 0.]]
```

```
# PyTorch Conv1d
conv = nn.Conv1d(1, 1, kernel_size=h_np.size, stride=stride, bias=False)
with torch.no_grad():
    conv.weight[:] = torch.tensor(h_np).view(1, 1, -1)

x_t = torch.tensor(x_np).view(1, 1, -1)
y_torch = conv(x_t).detach().numpy().squeeze()
```

```
# Compare results
print(f'Numpy:    y = {y_numpy}')
print(f'W matrix: y = {y_W}')
print(f'PyTorch:  y = {y_torch}')
```

```
Numpy:    y = [ 1.  1. -1. -1.]
W matrix: y = [ 1.  1. -1. -1.]
PyTorch:  y = [ 1.  1. -1. -1.]
```

```
print(f'x = {x_np}')
```

```
x = [ 0.  1.  2.  1.  0. -1. -2. -1.  0.]
```

Downsampling Without Filtering, hurting Shift Equivariance

```
# Use deterministic kernel to make comparisons transparent.
x_np = np.array([0., 1., 2., 1., 0., -1., -2., -1., 0.], dtype=np.float32)
#h_np = 1/4*np.array([1, 2, 1], dtype=np.float32)
h_np = np.array([1, 0, 0], dtype=np.float32)
#h_np = np.array([0, 1, 0], dtype=np.float32)
stride = 2
```

Stride 2 (Downsample with $M = 2$)
Size 1 Kernel, Phase 0

```
# NumPy: correlation convention to match PyTorch Conv1d.
full_valid = np.correlate(x_np, h_np, mode="valid")
y_numpy = full_valid[::stride]
```

```
# Weight matrix multiply
m = int(1 + np.ceil((x_np.size - h_np.size) / stride))  # number of rows
W = np.zeros((m, (m-1)*stride + h_np.size))
for i in range(m):
    W[i, i*stride:i*stride + h_np.size] = h_np
W = W[:,:x_np.size]
print(f'W =\n{W}')
y_W = W @ x_np
```

```
W =
[[1. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 1. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 1. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 1. 0. 0.]]
```

```
# PyTorch Conv1d
conv = nn.Conv1d(1, 1, kernel_size=h_np.size, stride=stride, bias=False)
with torch.no_grad():
    conv.weight[:] = torch.tensor(h_np).view(1, 1, -1)

x_t = torch.tensor(x_np).view(1, 1, -1)
y_torch = conv(x_t).detach().numpy().squeeze()
```

```
# Compare results
print(f'Numpy:    y = {y_numpy}')
print(f'W matrix: y = {y_W}')
print(f'PyTorch:  y = {y_torch}')
```

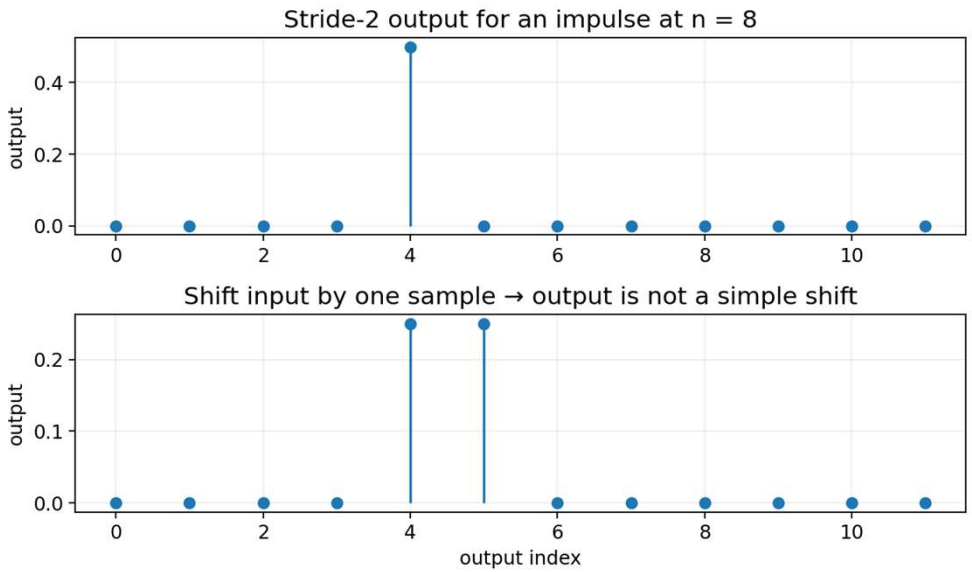
```
Numpy:    y = [ 0.  2.  0. -2.]
W matrix: y = [ 0.  2.  0. -2.]
PyTorch:  y = [ 0.  2.  0. -2.]
```

```
print(f'x = {x_np}')
```

```
x = [ 0.  1.  2.  1.  0. -1. -2. -1.  0.]
```

0:57–1:04

Why aliasing can hurt shift equivariance



- **Convolution alone** is shift equivariant (ignoring boundaries).
- **Subsampling** chooses a sampling phase.
- **A one-sample input shift** can move energy to a different phase before ↓M.
- **Aliased high-frequency feature content** makes the output vary sharply under small shifts.

Practical symptom

Predictions or feature maps can change more than expected after tiny translations or time shifts.

M=2

phase 0

phase 1

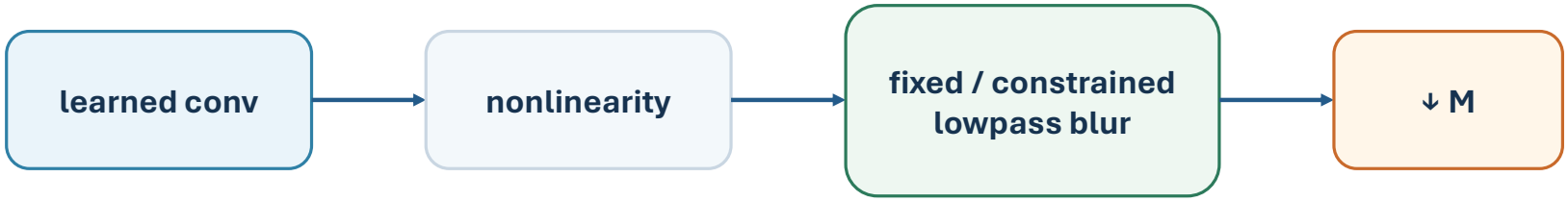
1 0 0 1 1 1 0 0 1 0

0 0 1 0 0

0 1 1 0 0

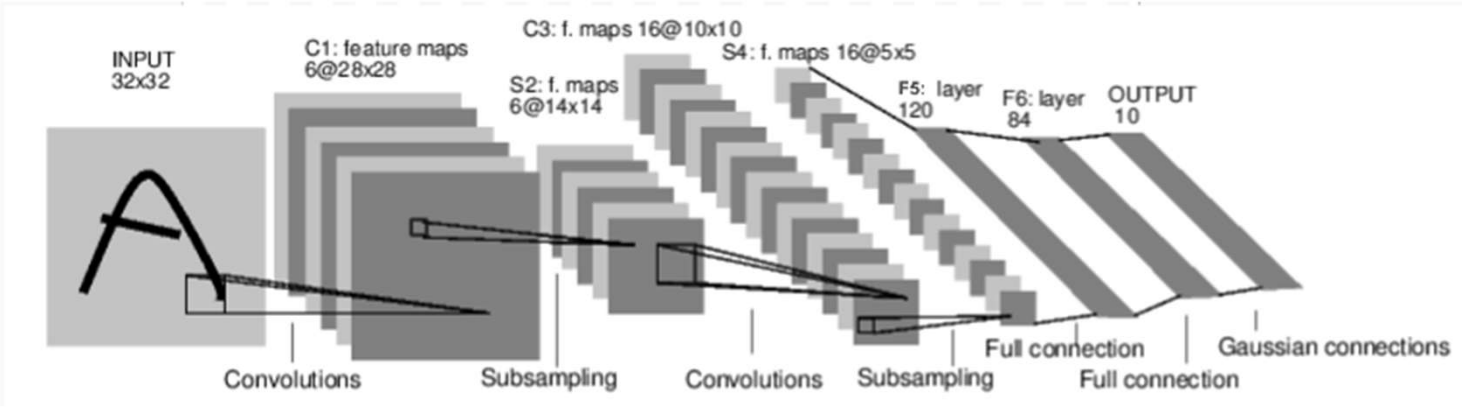
1:04–1:10

Anti-aliased CNNs: restore the missing DSP step



- BlurPool-style idea: lowpass before spatial or temporal subsampling.
- Alternative: constrain the strided kernel to have stronger lowpass behavior.
- Tradeoff: suppressing high-frequency detail can reduce aliasing but may also remove task-relevant features.

CNN for Classifying Digit Images



1:10–1:14

Mini-demo / coding prompt

Compare three stride-2 pipelines on the same 1-D signal:

A. Raw subsampling

y = x[::2]

B. Learned stride-2 conv

torch.nn.Conv1d(..., stride=2)

C. Blur + stride

lowpass(x) → take every 2nd sample

Shift the input by one sample. Which pipeline changes least, and why?

Use the accompanying notebook

Sections 1–5 reproduce the DSP spectra and the PyTorch stride comparison.

1:14–1:15

Lecture 13 wrap-up

- **Downsampling** changes the sample rate and stretches spectral copies.
- **Aliasing** occurs when those copies overlap.
- **Decimation** is anti-alias lowpass filtering followed by ↓M.
- **Strided CNN layers** are learned-filter + downsample systems.
- **Anti-aliased CNNs** reintroduce an explicit lowpass step before subsampling.

Exit question

In one sentence: why can a perfectly reasonable learned convolution still be a poor anti-aliasing filter?

Next lecture

Zero insertion creates images; interpolation removes them.